

Computer Vision

⇒ Interpolation

① → Nearest neighbour interpolation

Example

10	40
30	20

enlarge by factor of 5

10					40
	t				
		u		w	
					v
30					20

Find t, u, w, v ✓

$$t = 10$$

$$u = 30$$

$$w = 40$$

$$v = 20$$

As these were the original pixel values
closest to them.

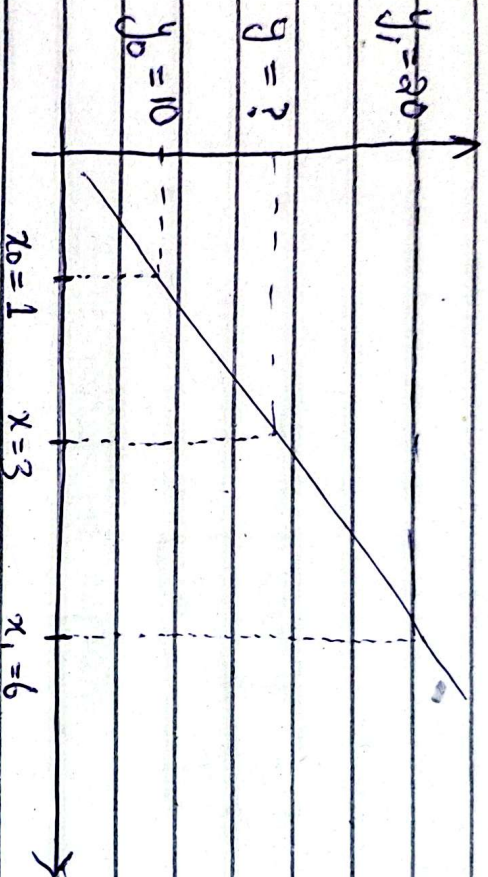
② Linear Interpolation.

Example: Find value of $f(3)$ using
linear interpolation, given $f(1)=10$ and
 $f(6)=20$.

$$f(1) = 10 ; x_0 = 1, y_0 = 10 \quad \checkmark$$

$$f(6) = 20 ; x_1 = 6, y_1 = 20 \quad \checkmark$$

$$f(3) = ? ; x = 3, y = ? \quad \checkmark$$



We know that formula of slope is:-

$$m = \frac{y_1 - y_0}{x_1 - x_0} \quad \checkmark$$

We can make two equations of slope using this because slope always remain the same.

$$m_1 = m_2$$

$$\frac{y_1 - y_0}{x_1 - x_0} = \frac{y_1 - y_0}{x_1 - x_0} \quad \checkmark$$

put values

$$\frac{y_1 - 10}{3 - 1} = \frac{20 - 10}{6 - 1} \quad \checkmark$$

$$\frac{y-10}{2} = \frac{10}{5}$$

$$y-10 = 2(2)$$

$$y = 10 + 4$$

$$y = 14$$

Example ::

10	40
30	20

↓ enlarged by factor of 5

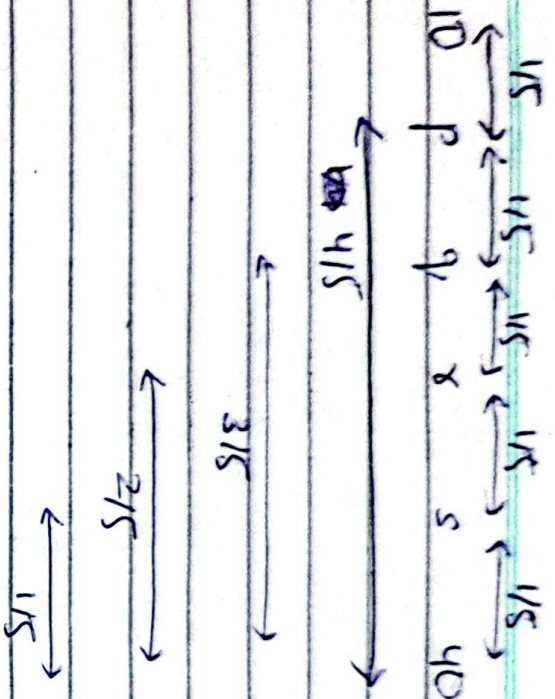
10	p	q	x	s	40
30					20

⇒ Find p, q, r, s by linear interpolation.

Suppose, distance between 10 and 40 in original image (2x2) is 1.

Suppose is enlarged, we say it is 1 to 5.

then distance from 10 to p is 1/5, from p to q is 1/5 and so on.



$$\text{Value at } p = \left(10 \times \frac{4}{5} + 40 \times \frac{1}{5}\right) \div 2$$

$$= \left(\frac{40}{5} + \frac{40}{5}\right) \div 2$$

$$= 80/5$$

$$p = 16$$

$$\text{Value at } q = \left(10 \times \frac{3}{5} + 40 \times \frac{2}{5}\right) \div 2$$

$$= \left(\frac{30}{5} + \frac{80}{5}\right) \div 2$$

$$= \frac{110}{5}$$

$$q = 22$$

$$\text{Value at } r = \left(10 \times \frac{2}{5} + 40 \times \frac{3}{5}\right) \div 2$$

$$= (20/5) + (120/5)$$

$$r = 28$$

$$\text{value at } S = (10 \times 1/5) + (40 \times 4/5) \approx 1$$

$$= \frac{10}{5} + \frac{160}{5}$$

$$= \frac{170}{5}$$

$$= 34$$

Example 4: Use bilinear interpolation to find values of t and u .

10	a	c			40
	t				
		u			
30	b	d			20

We can find values along the 'x' of t or along the 'y' of t to find " u " by linear interpolation.

So, let's mark pixel above t as ' a ' and pixel farthest below ' t ' as ' b '

First finding ' a ' and ' b ' like last example.

$$a = \left[\left(10 \times \frac{4}{5} \right) + \left(40 \times \frac{1}{5} \right) \right] \approx 1$$

$$a = 16$$

$$b = \left\lfloor \left(30 \times \frac{4}{5} \right) + \left(20 \times \frac{1}{5} \right) \right\rfloor \approx 1$$

$$b = 28$$

Now we can find 't' as we know a and b.

$$t = \left\lfloor \left(16 \times \frac{4}{5} \right) + \left(28 \times \frac{1}{5} \right) \right\rfloor \approx 1$$

$$t = 18.4$$

Now for "u" :-

lets label pixel in top of y-axis above 'u' as 'c' and bottom as 'd'

$$c = 22$$

$$d = 26$$

So,

$$u = \left\lfloor \left(22 \times \frac{2}{5} \right) + \left(26 \times \frac{3}{5} \right) \right\rfloor \approx 1$$

$$u = 26.4$$

So, basically value of a pixel depends on 'u' nearest neighbours in bilinear interpolation.

Bicubic interpolation :-

⇒ Uses 16 surrounding pixels 4×4

Toy example :

lets take a 4×4 pixel grid.
we want interpolator inside this region.

y \ x	x			
	0	1	2	3
0	10	20	20	40
1	15	25	35	45
2	20	30	40	50
3	25	35	45	55

Step 1 - write polynomial

$$P(x, y) = \sum_{i=0}^3 \sum_{j=0}^3 a_{ij} x^i y^j$$

Step 2 : Form / solve this equation

$$P(x, y) = a_{00} + a_{01}y + a_{02}y^2 + a_{03}y^3 +$$

$$a_{10}x + a_{11}xy + a_{12}xy^2 + a_{13}xy^3 +$$

$$a_{20}x^2 + a_{21}x^2y + a_{22}x^2y^2 + a_{23}x^2y^3 +$$

$$a_{30}x^3 + a_{31}x^3y + a_{32}x^3y^2 + a_{33}x^3y^3$$

Step-3 : Write equations from pixels:-

$$\rightarrow p(0,0) = a_{00}$$

Since $p_{00} = 10$

$$p(0,0) = 10 \Rightarrow \boxed{a_{00} = 10}$$

$$\rightarrow p(0,1) = a_{00} + a_{01} + a_{02} + a_{03}$$

~~we know~~ we know, $p(0,1) = 15$

$$\boxed{a_{00} + a_{01} + a_{02} + a_{03} = 15}$$

$$\rightarrow p(0,2) = a_{00} + 2a_{01} + 4a_{02} + 8a_{03}$$

$$\boxed{20 = a_{00} + 2a_{01} + 4a_{02} + 8a_{03}}$$

$$\rightarrow p(0,3) = a_{00} + 3a_{01} + 9a_{02} + 27a_{03}$$

$$\boxed{25 = a_{00} + 3a_{01} + 9a_{02} + 27a_{03}}$$

$$\rightarrow p(1,0) = a_{00} + a_{10} + a_{20} + a_{30}$$

$$\boxed{20 = a_{00} + a_{10} + a_{20} + a_{30}}$$

$$\rightarrow p(1,1) = a_{00} + a_{01} + a_{02} + a_{03} + a_{10} + a_{11} +$$

$$a_{12} + a_{13} + a_{20} + a_{21} + a_{22} + a_{23} + a_{30} + a_{31} + a_{32} + a_{33}$$

$$25 = a_{00} + a_{01} + a_{02} + a_{03} + a_{10} + a_{11} + a_{12} + a_{13} + a_{20} + a_{21} + a_{22} + a_{23} + a_{30} + a_{31} + a_{32} + a_{33}$$

$$\rightarrow P(1,2) = a_{00} + 2a_{01} + 4a_{02} + 8a_{03} + a_{10} + 2a_{11} + 4a_{12} + 8a_{13} + a_{20} + 2a_{21} + 4a_{22} + 8a_{23} + a_{30} + 2a_{31} + 4a_{32} + 8a_{33}$$

$$\rightarrow P(1,3) = a_{00} + 3a_{01} + 9a_{02} + 27a_{03} + a_{10} + 3a_{11} + 9a_{12} + 27a_{13} + a_{20} + 3a_{21} + 9a_{22} + 27a_{23} + a_{30} + 3a_{31} + 9a_{32} + 27a_{33}$$

$$\rightarrow P(2,0) = a_{00} + 2a_{10} + 4a_{20} + 8a_{30}$$

$$\rightarrow P(2,1) = a_{00} + a_{01} + a_{02} + a_{03} + 2a_{10} + 2a_{11} + 2a_{12} + 2a_{13} + 4a_{20} + 4a_{21} + 4a_{22} + 4a_{23} + 8a_{30} + 8a_{31} + 8a_{32} + 8a_{33}$$

$$\rightarrow P(2,2) = a_{00} + 2a_{01} + 4a_{02} + 8a_{03} + 4a_{10} + 4a_{11} + 8a_{12} + 16a_{13} + 4a_{20} + 8a_{21} + 16a_{22} + 32a_{23} + 8a_{30} + 16a_{31} + 32a_{32} + 64a_{33}$$

$$\rightarrow P(2,3) = a_{60} + 3a_{61} + 9a_{62} + 18a_{63} +$$

$$45$$

$$+ 2a_{10} + 6a_{11} + 18a_{12} + 54a_{13} +$$

$$4a_{20} + 12a_{21} + 36a_{22} + 108a_{23} +$$

$$8a_{30} + 24a_{31} + 72a_{32} + 216a_{33}$$

$$\rightarrow P(3,0) = a_{60} + 3a_{10} + 9a_{20} + 27a_{30}$$

$$45$$

$$\rightarrow P(3,1) = a_{60} + a_{01} + a_{02} + a_{03} +$$

$$45$$

$$3a_{10} + 3a_{11} + 3a_{12} + 3a_{13} +$$

$$9a_{20} + 9a_{21} + 9a_{22} + 9a_{23} +$$

$$27a_{30} + 27a_{31} + 27a_{32} + 27a_{33}$$

$$\rightarrow P(3,2) = a_{60} + 2a_{01} + 4a_{02} + 8a_{03} +$$

$$SD$$

$$3a_{10} + 6a_{11} + 12a_{12} + 24a_{13} +$$

$$9a_{20} + 18a_{21} + 36a_{22} + 72a_{23} +$$

$$27a_{30} + 54a_{31} + 108a_{32} + 216a_{33}$$

$$\rightarrow P(3,3) = a_{60} + 3a_{01} + 9a_{02} + 27a_{03} +$$

$$SS$$

$$3a_{10} + 9a_{11} + 27a_{12} + 81a_{13} +$$

$$9a_{20} + 27a_{21} + 81a_{22} + 243a_{23} +$$

$$27a_{30} + 81a_{31} + 108a_{32} + 729a_{33}$$

$$\text{Now, } AX = B$$

$$\text{or } X = A^{-1}B$$

$$X = A^{-1}B$$

$$\begin{bmatrix} a_{00} \\ a_{01} \\ a_{02} \\ a_{03} \\ a_{10} \\ a_{11} \\ a_{12} \\ a_{13} \\ a_{20} \\ a_{21} \\ a_{22} \\ a_{23} \\ a_{30} \\ a_{31} \\ a_{32} \\ a_{33} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}^{-1}$$

$$B = \begin{bmatrix} 10 & 15 & 20 & 25 & 20 & 25 & 30 & 35 & 20 & 35 \end{bmatrix}$$

$$\begin{bmatrix} 40 & 45 & 40 & 45 & 50 & 55 \end{bmatrix}^T$$



U-NET

⇒ Discrepancies

- (i) we remove some pooled layers to make 56^2 .
- (ii) the true on output layer is for classification object.

Since we are doing pooling at each stage, so we will have to remove those layers at each stage as well is upward part of "U".

For a binary class problem, 1 neuron can be used for probability based thresholding.

We can also use 2 neurons for 2 classes.

In a tracking based system:-

System should be invariant to:-

- (i) Scale
- (ii) Translation
- (iii) Rotation

Scale: close to camera go far away. System should be able to detect. Usually there is a range.

Translation: movement along x and y axis or in between should be catered.
(could be catered by template cutting at motor, latter)

Rotation & spinning deformity

⇒ Resizing to a smaller size helps

Let's say scale is 1.1-1.4

translated : $2 \times 640 = 1280$ (640 x 640)
image

Rotation : $0 \leq \theta < 180$

Let's say we want
angle at 0.25° intervals

$$180 \times 4 = 720$$

$$\text{total iterations} = 40 \times 1280 \times 720$$

→ For resizing, we generally don't go below 5 levels.

e.g. $128 \rightarrow 64 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow 4$

For non-deformable objects we use normalized
- correlation

For resizing → pyramid creation

For translation → template matching.

⇒ Subsampling and Aliasing:

Gaussian → ideal → but it is gaussian is
both time and frequency
domain.

$$\left[\begin{array}{cccccc} 0.0625 & 0.25 & 0.375 & 0.25 & 0.0625 \end{array} \right]$$

$$\rightarrow \frac{1}{16} \left[\begin{array}{cccccc} 1 & 4 & 6 & 4 & 1 \end{array} \right]$$

Using gaussian before downsample remove the checkerboard pattern on the image.

Gaussian pyramid operators are normalized from center and corners will not be filtered right but we do not usually care for corners.

Other matrix $\rightarrow$ remove a row and column after a row and column.

Low pass filter goes to transition from high to low. save sub which has more parts, opposite to high pass filter.

Isotropic and non-isotropic filters.

$$[sR + 1]$$

s: scale

R: rotate

t: translate

We will make separable filters for rotation.

Computes Viterbi

$$\begin{bmatrix} 1 & -1 \end{bmatrix} \rightarrow \{G=0\} \Rightarrow H.P$$

$$\begin{bmatrix} 1 & 1 \end{bmatrix} \rightarrow \{G>0\} \rightarrow L.P$$

(~~would be~~ ~~trans pass or~~
b and step since positive
sum -

If all coefficients of
a filter are positive,
it is low pass.)

1st derivative filter: $f(x+1) - f(x)$

2nd derivative filter: $f(x+1) + f(x-1) - 2f(x)$

If we want to take 3rd derivative from first order
elements:

$$f(x+1) - f(x)$$

$$\ominus f(x) \oplus f(x-1)$$

$$f(x+1) + f(x-1) - 2f(x)$$

Laplacian:-

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix} \quad \begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

$$I + \Delta(I)$$

$$I - \Delta(I)$$

Boundary
sharpened

Boundary
unsharpened

$$f + c \nabla^2 f$$

$$f - c \nabla^2 f$$

high
boost $\leftarrow$

$\rightarrow$ unsharp

"c" is controlling factor

for sharpening.

In frequency domain:-

$$F \left[\frac{d^2 f}{dx^2} \right] = -(2\pi u)^2 F(u, v)$$

$$F \left(\frac{d^2 f}{dy^2} \right) = -(2\pi v)^2 F(u, v)$$

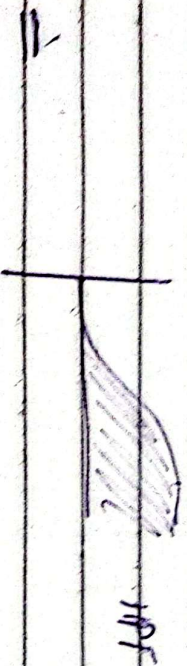
$$\text{filter} = -[(2\pi u)^2 + (2\pi v)^2] F(u, v)$$



Yes you can pass pass pass (0,0)

but since Laplace is high filter, we use mod when using filter in frequency domain

$$\text{filter} = \sqrt{-(\omega a)^2 + (2\pi v)^2 / \text{Filter}} /$$



⇒ Introduction to the Fourier Transform and the Frequency Domain.

$$f(x) = [1 \ 2 \ 3 \ 4 \ 5 \ 6 \ 7 \ 8]$$

$$x = [0 \ 1 \ 2 \ 3 \ 4 \ 5 \ 6 \ 7]$$

$$(-1)^x = [1, -1, 1, -1, 1, -1, 1, -1]$$

$$f(x) = [1, -2, 3, -4, 5, -6, 7, -8]$$

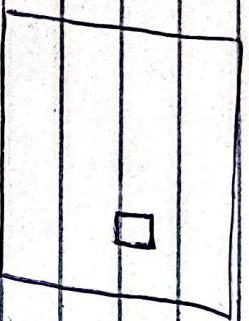
Blending:-

$$L_{\text{blend}}^k = M^k \cdot L_A^k + (1 - M^k) \cdot L_B^k$$

Computer Vision

D. Singh et al.

Fig 1 The RF CNN network architecture



$$I_m = I - \text{mod}(I)$$

remove smoothness/objects

⇒ DCT coefficients are random for natural images (no pattern for original images)

forged images will have some pattern in the DCT

$$\left[\begin{array}{c|c} N & -E \\ \hline S & 1 \end{array} \right]$$

⇒ ANN vs CNN

why CNN for images: spatial info is retained in CNN

ANN would be more computationally expensive than CNN because of flattening.

What is batch normalization in images.

⇒ search layer covariance.

→ see calculation for parameters and memory.

First layer k with self memory associated hogi, not parameters.

Parameters come into play when dealing with weights.

Also in Max-Pooling, there is no learning of weights so it has zero parameters.

For parameters, there must be learnable parameters.

without biases

$$(3 \times 3 \times 3) \times 64$$

with biases

$$[(3 \times 3 \times 1) \times 3] \times 64$$

Dropout can be used b/w FCs to reduce overfitting and improve generalization.

To make directional filter:-

$$T = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix} \quad I_{xx} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & -2 & 1 \\ 0 & 0 & 0 \end{bmatrix}$$

change along X-axis

$$I_{yy} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & -2 & 0 \\ 0 & 1 & 0 \end{bmatrix} \quad I_{xy} = \begin{bmatrix} -1 & 0 & 1 \\ 0 & 0 & 0 \\ 1 & 0 & -1 \end{bmatrix}$$

change along X-axis

$$\Delta\theta = I_{xx} \cos^2\theta + 2 I_{xy} \sin\theta \cos\theta + I_{yy} \sin^2\theta$$

for 15° using $\Delta\theta$:

$$\begin{bmatrix} -0.12 & 0.07 & 0.12 \\ 0.93 & -2 & 0.93 \\ 0.12 & 0.07 & -0.12 \end{bmatrix}$$